

INDEX

Name: N. SREEKAR REDDY

Subject : Operating Systems

Std : _____ Sec. : 4-D

Roll No. :

Sl No.	Date	Title	Marks	Teacher's Sign/Remarks
1.	6/3/26	FCFS & SJF	(10)	8/3 6/3
2.	13/3/26	Priority (Preemptive & non-Preemptive)	(10)	8/3
3.	27/3/26	Round robin scheduling & multilevel feedback queue	(10)	27/3
4.	10/4/26	RMS & EDF Scheduling	(10)	8/3
5.	24/4/26	Bounded Buffer & Dining Philosopher	(10)	8/3
6.	8/5/26	a) Banker's algorithm b) deadlock detection	(10)	8/5
7.	15/5/26	Memory allocation techniques	10	8/5
8.	22/5/26	Page Replacement algorithms		

Q. Write a C program to find the Second Smallest element in an array.

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
int main() {
```

```
    int n, arr[100];
```

```
    int smallest = INT_MAX, SecondSmallest = INT_MAX;
```

```
    printf("Enter number of elements: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter elements: \n");
```

```
    for(int i=0; i<n; i++) {
```

```
        scanf("%d", &arr[i]);
```

```
        if (arr[i] < smallest) {
```

```
            SecondSmallest = smallest;
```

```
            smallest = arr[i];
```

```
        }
```

```
        else if (arr[i] < SecondSmallest && arr[i] !=  
                smallest) {
```

```
            SecondSmallest = arr[i];
```

```
        }
```

```
    }
```

```
    if (SecondSmallest == INT_MAX)
```

```
        printf("No Second Smallest element. \n");
```

```
    else
```

```
        printf("Second smallest = %d \n",
```

```
                SecondSmallest);
```

```
    return 0;
```

```
}
```

2. Write a program in C to find the sum of left diagonals of a matrix.

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, a[10][10], sum = 0;
```

```
    printf("Enter order of square matrix:");
```

```
    scanf("%d", &n);
```

```
    printf("Enter matrix elements:\n");
```

```
    for(int i=0; i<n; i++)
```

```
        for(int j=0; j<n; j++)
```

```
            scanf("%d", &a[i][j]);
```

```
    for(int i=0; i<n; i++)
```

```
        sum += a[i][i];
```

```
    printf("Sum of left diagonal = %d\n", sum);
```

```
    return 0;
```

```
}
```

output

Enter order of square matrix: 2

Enter matrix elements:

2

4

6

8

Sum of left diagonal = 10

3. Write a program in c to find the Sum of rows and columns of a matrix.

```
#include <stdio.h>
```

```
int main() {
```

```
    int r, c, a[10][10];
```

```
    printf("Enter rows and columns:");
```

```
    scanf("%d %d", &r, &c);
```

```
    printf("Enter matrix:\n");
```

```
    for(int i=0; i<r; i++)
```

```
        for(int j=0; j<c; j++)
```

```
            scanf("%d", &a[i][j]);
```

```
    for(int i=0; i<r; i++) {
```

```
        int rowSum=0;
```

```
        for(int j=0; j<c; j++)
```

```
            rowSum+=a[i][j];
```

```
        printf("Sum of row %d = %d\n", i+1,
```

```
            rowSum);
```

```
    }
```

```
    for(j=0; j<c; j++) {
```

```
        int colSum=0
```

```
        for(int i=0; i<r; i++)
```

```
            colSum+=a[i][j];
```

```
        printf("Sum of column %d = %d\n", j+1,
```

```
            colSum)
```

```
    }
```

```
    return 0;
```

```
}
```

output:

Enter no of rows in matrix

2
2
1
2

$\begin{bmatrix} 1 & 2 \\ 1 & 2 \end{bmatrix}$

matrix is

1 2
1 2

row sums:

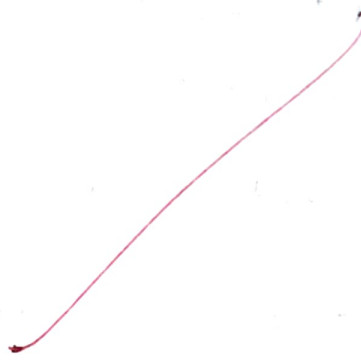
3

3

column sum

2

4



4. Write a program in C to Count the total number of duplicate elements in an array.

```
#include <stdio.h>

int main() {
    int n, arr[100], count = 0;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
        for (int j = 0 i + 1; j < n; j++) {
            if (arr[i] == arr[j]) {
                count++;
                break;
            }
        }
    }

    printf("Total duplicate elements = %d\n", count);
    return 0;
}
```

output

Enter no of element in array 5

5

5

2

1

3

Total no of duplicate elements = 1

5. Write a program in c to find the second largest / element in an array.

```
#include <stdio.h>
```

```
int main() {
```

```
    printf("Enter the number of elements in  
           the array: ");
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int arr[20];
```

```
    for (int i = 0; i < n; i++) {
```

```
        scanf("%d", &arr[i]);
```

```
    }
```

```
    int b1 = -1000;
```

```
    int b2 = -1000;
```

```
    for (int j = 0; j < n; j++) {
```

```
        scanf("%d", &arr[j]);
```

```
        if (arr[j] > b1) {
```

```
            b2 = b1;
```

```
            b1 = arr[j];
```

```
        }
```

```
        else if (b2 < arr[j] && b1 != arr[j]) {
```

```
            b2 = arr[j];
```

```
        }
```

```
    }
```

```
    printf("The second smallest element from  
           array is %d", b2);
```

output :-

enter number of element in array 5

1

2

3

4

5

The second smallest from the array is 2

Q6) WAP in C to delete an element at a desired position from an array.

```
#include <stdio.h>
```

```
int main(){
```

```
    printf("Enter number of elements in  
        array");
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int arr[20];
```

```
    for(int i=0; i<n; i++){
```

```
        scanf("%d", &arr[i]);
```

```
    }
```

```
    int pos;
```

```
    printf("Enter pos of element to  
        delete:");
```

```
    scanf("%d", &pos);
```

```
    if(pos < 0 || pos > n){
```

```
        printf("Invalid Pos");
```

```
    }
```

```
    for(int j = pos; j < n-1; j++){
```

```
        arr[j] = arr[j+1];
```

```
    }
```

```
    printf("The new array is!");
```

```
    for(int i=0; i<n-1; i++){
```

```
        printf("%d", arr[i]);
```

```
    }
```

```
    return 0;
```

Output:

enter number of element in array 6

2
3
5
5
6
7

enter pos of element to delete 1
new array 2
5
6
7

6/3/26

Q. Implement FCFS using C

```
#include <stdio.h>
```

```
int main(){
```

```
    int n, i;
```

```
    int at[20], bt[20], wt[20], tat[20], ct[20];
```

```
    float avg-wt = 0, avg-tat = 0;
```

```
    printf("Enter number of processes:");
```

```
    scanf("%d", &n);
```

```
    printf("Enter Arrival Time and Burst Time:\n");
```

```
    for(i=0; i<n; i++){
```

```
        printf("P%d Arrival Time", i+1);
```

```
        scanf("%d", &at[i]);
```

```
        printf("P%d Burst Time:", i+1);
```

```
        scanf("%d", &bt[i]);
```

```
    }
```

```
    ct[0] = at[0] + bt[0];
```

```
    for(int i=1; i<n; i++) {
```

```
        if (ct[i-1] < at[i])
```

```
            ct[i] = at[i] + bt[i];
```

```
        else
```

```
            ct[i] = ct[i-1] + bt[i];
```

```
    }
```

```
    for(i=0; i<n; i++){
```

```
        tat[i] = ct[i] - at[i];
```

```
        wt[i] = tat[i] - bt[i];
```

```
        avg-wt += wt[i];
```

```
        avg-tat += tat[i];
```

```

3
printf("\n Process\t tAT\t BT\t WT\t TAT\n");
for(i=0; i<n; i++) {
    printf("P%d\t %d\t %d\t %d\t %d\t %d\n",
        i+1, at[i], bt[i], ct[i], wt[i],
        tat[i]);
}

printf("\n Average Waiting Time= %.2f", avg-wt/n);
printf("\n Average Turnaround Time= %.2f",
    avg-tat/n);

return 0;
}

```

output

Process	AT	BT	CT	TAT	WT
P1	2				
P2	8				
P3	4				

Enter number of processes : 4

Enter Arrival Time and Burst Time:

P1 Arrival Time: 0

P1 Burst Time: 5

P2 Arrival Time: 1

P2 Burst Time: 3

P3 Arrival Time: 2

P3 Burst Time: 8

P4 Arrival Time: 3

P4 Burst Time: 6

Process	AT	BT	CT	TAT	WT	RT
P1	2	5	5	5	0	0
P2	1	3	8	7	4	4
P3	2	8	16	14	6	6
P4	3	6	22	19	13	13
Average waiting Time = 5.75 5.75						

Average Turnaround Time = ~~11.25~~ 11.25

RT = ?

8/13

2. SJF (Preemptive & non preemptive).

```
#include <stdio.h>
```

```
int main()
```

```
void swap(int *a, int *b){
```

```
    int temp = *a;
```

```
    *a = *b;
```

```
    *b = *temp;
```

```
int main(){
```

```
    int n, i, j, choice;
```

```
    int P[20], AT[20], BT[20], CT[20], TAT[20],
```

```
    WT[20], RT[20];
```

```
    float avg-wt = 0, avg-tat = 0;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter arrival time and burst time  
    for each process:\n");
```

```
    for(i = 0; i < n; i++){
```

```
        printf("P%d Arrival Time: ", i+1);
```

```
        scanf("%d", &AT[i]);
```

```
        printf("P%d Burst Time: ", i+1);
```

```
        scanf("%d", &BT[i]);
```

```
        P[i] = i+1;
```

```
        RT[i] =
```

```

}
printf("\n Choose Scheduling Type : \n");
printf("1. Non-preemptive SJF \n");
printf("2. Preemptive SJF (SRTF) \n");
printf("Enter choice : ");
scanf("%d", &choice);
switch(choice) {

```

```

    case 1: {

```

```

        int time = 0, completed[20] = {0},
        count = 0;

```

```

        while(count < n) {

```

```

            int idx = -1, min_bt = 9999;

```

```

            for(i = 0; i < n; i++) {

```

```

                if (!completed[i] && AT[i] <= time &&

```

```

                    BT[i] < min_bt) {

```

```

                    min_bt = BT[i];

```

```

                    idx = i;
                }
            }

```

```

            if (idx == -1) {

```

```

                time++;
            }

```

```

            else {

```

```

                time += BT[idx];

```

```

                CT[idx] = time;

```

```

                Completed[idx] = 1;

```

```

                count++;
            }

```

```

        }

```

```

        }

```

```

        break;
    }
}

```

}

Case 2: {

int time = 0, completed = 0;

while (completed < n) {

int idx = -1, min_rt = 9999;

for (i = 0; i < n; i++) {

if (AT[i] <= time && RT[i] > 0 && RT[i] < min_rt)

min_rt = RT[i];

idx = i;

}

if (idx == -1) {

time++;

} else {

RT[idx]--;

time++;

if (RT[idx] == 0) {

CT[idx] = time;

Completed++;

}

}

}

break;

}

default:

printf("Invalid choice!\n");

return 0;

}

for (i = 0; i < n; i++) {

TAT[i] = CT[i] - AT[i];

WT[i] = TAT[i] - BT[i];

avg_wt += WT[i];

avg_tat += TAT[i];

```

}
avg-wt /= n;
avg-tat /= n;
printf("\n Process\tAT\tBT\tCT\tTAT\tWT\n");
for (i=0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i], AT[i], BT[i], CT[i], TAT[i],
        WT[i]);
}

printf("\n Average Waiting Time: %.2f", avg-wt);
printf("\n Average Turnaround Time: %.2f\n",
    avg-tat);

return 0;
}

```

Output:

Enter number of processes: 4

Enter arrival Time and burst time for each Process:

P1 arrival time: 0

P1 burst time: 5

P2 arrival time: 1

P2 burst time: 3

P3 arrival time: 2

P3 burst time: 8

P4 arrival time: 3

P4 burst time: 6

choose scheduling Type:

1. Non Preemptive
2. Preemptive
- 1.

Process	AT	BT	WT	TAT
P1	0	5	3	8
P2	1	3	0	3
P3	2	8	12	20
P4	3	6	5	11

Arg waiting time: 5

2. Arg TAT: 10.5

Process	AT	BT	WT	TAT
P2	1	3	6	3
P1	0	5	3	8
P4	3	6	8	14
P3	2	8	14	22

Arg waiting time: 7.75

Arg TAT: 11.75

By comparing all SJF non preemptive
has shortest waiting time.


9/3/26

2. Priority non preemptive

```
#include <stdio.h>
```

```
struct process {
```

```
int id, bt, at, pr, ct, tat, wt;
```

```
};
```

```
int main() {
```

```
struct process p[10], temp;
```

```
int n, i, j;
```

```
printf("Enter number of processes:");
```

```
scanf("%d", &n);
```

```
for(int i=0; i<n; i++)
```

```
{
```

```
printf("\n Process %d\n", i+1);
```

```
printf("Arrival Time:");
```

```
scanf("%d", &p[i].at);
```

```
printf("Burst Time:");
```

```
scanf("%d", &p[i].bt);
```

```
printf("Priority:");
```

```
scanf("%d", &p[i].pr);
```

```
p[i].id = i+1;
```

```
}
```

```
for(i=0; i<n; i++) {
```

```
for(j=i+1; j<n; j++) {
```

```
if (p[i].pr > p[j].pr) {
```

```
temp = p[i];
```

```
p[i] = p[j];
```

```
p[j] = temp;
```

```
    }  
}
```

```
int time = 0;
```

```
for(i=0; i<n; i++){
```

```
    if(time < p[i].at)
```

```
        time = p[i].at;
```

```
    p[i].ct = time + p[i].bt;
```

```
    time = p[i].ct;
```

```
    p[i].tat = p[i].ct - p[i].at;
```

```
    p[i].wt = p[i].tat - p[i].bt;
```

```
}
```

```
printf("\nID\tAT\tBT\tPR\tCT\tTAT\tWT\n");
```

```
for(i=0; i<n; i++){
```

```
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n", p[i].id, p[i].
```

```
        p[i].bt, p[i].pr, p[i].ct, p[i].tat,
```

```
        p[i].wt);
```

```
}
```

```
}
```

Output:-

Enter number of processes: 3

Process: 1

Arrival Time: 0

Burst Time: 4

Priority: 2

Process 2

Arrival Time: 3

Burst Time: 3

Priority: 1

Process 3

Arrival Time: 4

Burst Time: 3

Priority: 3

Process	AT	BT	Pri	CT	TAT(CT-AT)	WT(TAT-BT)
1	0	4	2	10	10	6
2	3	3	1	6	3	0
3	4	3	3	13	9	6

$$\text{Avg TAT} = \frac{22}{3} = 7.33 \quad \text{Avg WT} = \frac{12}{3} = 4$$

* Preemptive

```
#include <stdio.h>
```

```
int main() {
```

```
int n, i, time = 0, completed = 0;
```

```
int bt[10], at[10], pr[10];
```

```
int rt[10], ct[10], tat[10], wt[10];
```

```
int highest, min-pr;
```

```
printf("Enter number of processes:");
```

```
scanf("%d", &n);
```

```
for(i = 0; i < n; i++)
```

```
printf("\n Process %d\n", i+1);
```

```
printf("Arrival Time: ");
```

```
scanf("%d", &at[i]);
```

```
printf("Burst Time: ");
```

```
scanf("%d", &bt[i]);
```

```
printf("Priority: ");
```

```
scanf("%d", &pr[i]);
```

```
rt[i] = bt[i];
```

```
}
```

```
while (completed != n) {
```

```
    highest = -1
```

```
    min-pr = 999;
```

```
    for (i = 0; i < n; i++) {
```

```
        if (at[i] <= time && rt[i] > 0 && pr[i] < min-pr) {
```

```
            min-pr = pr[i];
```

```
            highest = i;
```

```
        }
```

```
    }
```

```
    if (highest == -1) {
```

```
        time++;
```

```
        continue;
```

```
    }
```

```
    rt[highest]--;
```

```
    time++;
```

```
    if (rt[highest] == 0) {
```

```
        completed++;
```

```
        at[highest] = time;
```

$tat(highest) = ct(highest) - at(highest);$

$wt(highest) = tat(highest) - bt(highest);$

}
}

printf("\n Process\tAT\tBT\tCT\tAT\tWT\n");

for(i=0; i<n; i++)

printf("Pr%d\t%d\t%d\t%d\t%d\t%d\n",

i+1, at[i], bt[i], pr[i], ct[i], wt[i]);

}

}

output:-

Enter number of processes: 3

Process 1

Arrival Time: 0

Burst Time: 4

Priority : 2

Process 2

Arrival Time: 3

Burst Time: 3

Priority: 1

Process: 3

Arrival time: 4

Burst Time: 3

Priority: 3

Process	AT	BT	Pri	CT	TAT	WT
1	0	4	2	7	7	3
2	3	3	1	6	3	0
3	4	3	3	10	6	3

$$\text{Avg TAT} = 5.34$$

$$\text{Avg WT} = 2$$

→ Here preemptive is the optimal solution as Turn Around Time and waiting time are less.

8/2
27/3

Q. Roundrobin Scheduling

```
#include <stdio.h>
```

```
Struct Process{
```

```
int pid, at, bt, ct, tat, wt, rt;
```

```
};
```

```
int main(){
```

```
int n, tq;
```

```
struct Process P[20];
```

```
int queue[100];
```

```
int visited[20] = {0};
```

```
int front = 0, rear = 0;
```

```
int time = 0, Completed = 0;
```

```
int order[200];
```

```
int timeMark[201];
```

```
int k = 0;
```

```
float avgwt = 0, avgtat = 0;
```

```
printf("Enter number of processes:");
```

```
scanf("%d", &n);
```

```
for(int i = 0; i < n; i++){
```

```
    P[i].pid = i + 1;
```

```
    printf("\nProcess %d\n", i + 1);
```

```
    printf("Arrival Time:");
```

```
    scanf("%d", &P[i].at);
```

```
    printf("Burst Time:");
```

```
    scanf("%d", &P[i].bt);
```

```
    P[i].rt = P[i].bt;
```

```
}
```

```
for (int i = 0; i < n-1; i++) {
```

```
    for (int j = i+1; j < n; j++) {
```

```
        if (P[i].at > P[j].at) {
```

```
            struct Process temp = P[i];
```

```
            P[i] = P[j];
```

```
            P[j] = temp;
```

```
        }
```

```
    }
```

```
}
```

```
printf("\nEnter Time Quantum: ");
```

```
scanf("%d", &tq);
```

```
timeMark[0] = 0;
```

```
while (completed < n) {
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (P[i].at <= time && visited[i] == 0) {
```

```
            queue[rear++] = i;
```

```
            visited[i] = 1;
```

```
        }
```

```
    }
```

```
    if (P[idx].rt > tq) {
```

```
        P[idx].rt -= tq;
```

```
        timer += tq;
```

```
    }
```

```
    else {
```

```
        timer += P[idx].rt;
```

```
        P[idx].rt = 0;
```

```
        P[idx].ct = timer;
```

```
        P[idx].tat = P[idx].ct - P[idx].at;
```

```
        P[idx].wt = P[idx].tat - P[idx].bt;
```

```
avgwt += p[idx].wt
```

```
avgtat += p[idx].tat;
```

```
Completed++;
```

```
}
```

```
k++;
```

```
timeMark[k] = time;
```

```
for(int i=0; i<n; i++){
```

```
    if (p[i].at <= time && visited[i] == 0) {
```

```
        queue[rear++] = i;
```

```
        visited[i] = 1;
```

```
    }
```

```
}
```

```
if (p[idx].rt > 0) queue[rear++] = idx;
```

```
}
```

```
printf("In Gantt chart: \n");
```

```
for(int i=0; i<k; i++) printf("%d", Order[i]);
```

```
printf("\n\n");
```

```
for(int i=0; i<k; i++) printf("%d-%d", timeMark[i],
```

```
printf("\n\n");
```

```
printf("Process\tAT\tBT\tCT\tTAT\tWT\n");
```

```
for(int i=0; i<n; i++){
```

```
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
```

```
        p[i].pid, p[i].at, p[i].bt, p[i].ct, p[i].tat,
```

```
        p[i].wt);
```

```
}
```

```
printf("\nAverage TAT = %.2f", avgtat/n);
```

```
printf("\nAverage WT = %.2f\n", avgwt/n);
```

```
return 0;
```

```
}
```

Output :-

Enter number of processes: 3

Process 1

Arrival Time: 0

Burst Time: 5

Process 2

Arrival Time: 1

Burst Time: 3

Process 3

Arrival Time: 2

Burst Time: 8

Enter Time Quantum: 2

-- Round Robin Scheduling ---

Gantt chart:

| P1 | P2 | P3 | P1 | P2 | P3 | P1 | P3 | P3 |
0 2 4 6 8 9 11 12 14 16

Process	AT	BT	CT	TAT	WT
P1	0	5	12	12	7
P2	1	3	9	8	5
P3	2	8	16	14	6

Avg TAT = 11.33

Avg WT = 6.00

Enter Time Quantum: 4

-- Round Robin Scheduling ---

Gantt chart:

| P1 | P2 | P3 | P1 | P3 |
0 4 7 11 12 16

Process	AT	BT	CT	TAT	WT
P1	0	5	12	12	7
P2	1	3	7	6	3
P3	2	8	16	14	6

Average TAT = 10.67

Average WT = 5.33

∴ If Time Quantum is more then waiting time is less.

Q. Multilevel feedback queue.

```
#include <stdio.h>
```

```
struct Process {int id, at, bt, type, ct, wt,  
                tat;};
```

```
int main() {
```

```
    int n;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    struct Process p[n], temp;
```

```
    for (int i = 0; i < n; i++) {
```

```
        p[i].id = i;
```

```
        printf("\n Process %d\n Enter arrival time:", i);
```

```
        scanf("%d", &p[i].at);
```

```
        printf("Enter burst time: ");
```

```
        scanf("%d", &p[i].bt);
```

```
        printf("Enter type (0 = System, 1 = user): ");
```

```
        scanf("%d", &p[i].type);
```

```
    }
```

```
    for (int i = 0; i < n - 1; i++)
```

```
for(int j = 0; j < n - i - 1; j++)
```

```
if (p[j].type > p[j+1].type || (p[j].type ==  
type && p[j].at > p[j+1].at)
```

```
temp = p[j];
```

```
p[j] = p[j+1];
```

```
p[j+1] = temp;
```

```
}
```

```
int time = 0; float twt = 0, ttat = 0;
```

```
printf("\nID\tType\tAT\tBT\tCT\tWT\tTAT\n")
```

```
for(int i = 0; i < n; i++) {
```

```
if (time < p[i].at) time = p[i].at;
```

```
time += p[i].bt;
```

```
p[i].ct = time;
```

```
p[i].tat = p[i].ct - p[i].at;
```

```
p[i].wt = p[i].tat - p[i].bt;
```

```
twt += p[i].wt; ttat += p[i].tat;
```

```
printf("%d\t%s\t%d\t%d\t%d\t%d\t%d\n",
```

```
p[i].id, p[i].type, "User": "System", p[i].at,
```

```
p[i].bt, p[i].ct, p[i].wt, p[i].tat);
```

```
}
```

```
printf("\nAverage Waiting Time: %.2f\nAverage
```

```
Turnaround Time: %.2f", twt/n,
```

```
ttat/n);
```

```
return 0;
```

```
}
```

output:-

Enter number of processes: 4

Process 0

Enter arrival time: 0

Enter burst time: 5

Enter type (0 = System, 1 = User): 0

Process 1

Enter arrival time: 2

Enter burst time: 3

Enter type (0 = System, 1 = user): 1

Process 2

Enter arrival time: 1

Enter burst time: 4

Enter type (0 = System, 1 = user): 0

Process 3

Enter arrival time: 3

Enter burst time: 2

Enter type (0 = System, 1 = user): 1

ID	Type	AT	BT	CT	WT	TAT
0	System	0	5	5	0	5
1	System	1	4	9	4	8
2	User	2	3	12	7	10
3	User	3	2	14	9	11

Average Waiting Time: 5.00

Average Turnaround Time: 8.50


27/3

10-4-26

Q. RMS and EDF Scheduling

```
#include <stdio.h>
```

```
#include <math.h>
```

```
#define MAX 20
```

```
#define MAX-TIME 1000
```

```
struct Task{
```

```
int id;
```

```
int bt;
```

```
int deadline;
```

```
int period;
```

```
int ct;
```

```
int wt;
```

```
int tat;
```

```
};
```

```
double utilization_edf(struct Task tasks[], int
```

```
double u=0;
```

```
for(int i=0; i<n; i++) u+= (double) tasks[i].bt/  
tasks[i].deadline;
```

```
return u;
```

```
}
```

```
double rm_bound(int n){
```

```
return n*(pow(2.0, 1.0/n)-1.0);
```

```
}
```

```
void print_chart(int schedule[], int total, char  
title[]){  
printf("\n%s Gantt Chart\n", title);  
printf("|");
```

```
for(int i=0; i<total; i++){
```

```
if(schedule[i]==0) printf("Idle");
```

```
else printf("P %.-2d", schedule[i]);
```

```
}
```

```
printf("\n");
```

```
for(int i=0; i<total; i++){
```

```
printf("%.6d", i+1);
```

```
}
```

```
printf("\n");
```

```
}
```

```
void edf (Struct Task tasks[], int n){
```

```
int time = 0;
```

```
int schedule[MAX_TIME]={0};
```

```
for(int i=0; i<n-1; i++){
```

```
for(int j = i+1; j<n; j++){
```

```
if(tasks[i].deadline > tasks[j].deadline){
```

```
struct Task tmp = tasks[i]; tasks[i] = tasks[j];
```

```
tasks[j] = tmp;
```

```
}
```

```
}
```

```
}
```

```
printf("\n=== Earliest Deadline First (EDF) Scheduling ===\n");
```

```
printf("%-10s %-5s %-8s %-5s %-5s %-5s\n", "ProcessID",
```

```
"BT", "Deadline", "CT",
```

```
"WT", "TAT");
```

```
for(int i=0; i<n; i++){
```

```
for(int k=time; k<time+tasks[i].bt; k++){
```

```
schedule[k] = tasks[i].id;
```

```
time+=tasks[i].bt;
```

```
tasks[i].ct = time;
```

```
tasks[i].tat = tasks[i].ct;
```

```

tasks[i].wt = tasks[i].tat - tasks[i].bt;
printf("%-10d %-5d %-8d %-5d %-5d %-5d\n",
    tasks[i].id, tasks[i].bt, tasks[i].deadline, tasks[i].ct,
    tasks[i].at, tasks[i].tat);
}

```

```

print_chart(schedule, time, "EDF");
double u = utilization_edf(tasks, n);
if (u <= 1.0) printf("EDF Result: schedulable
                    (u = %.2f)\n", u);
else printf("EDF Result: Not schedulable
            (u = %.2f)\n", u);
}

```

```

void rms(struct Task tasks[], int n) {
    int time = 0;
    int schedule[MAX_TIME] = {0};
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (tasks[i].period > tasks[j].period) {
                struct Task tmp = tasks[i];
                tasks[i] = tasks[j];
                tasks[j] = tmp;
            }
        }
    }
}

```

```

printf("\n == Rate Monotonic Scheduling (RMS) == \n");
printf("%-10s %-5s %-6s %-5s %-5s %-5s\n",
    "ProcessID", "BT", "Period", "CT", "WT", "TAT");

```

```

for (int i = 0; i < n; i++) {
    for (int k = time; k < time + tasks[i].bt; k++)
        schedule[k] = tasks[i].id;
    time += tasks[i].bt;
    tasks[i].ct = time;
    tasks[i].tat = tasks[i].ct;
    tasks[i].wt = tasks[i].tat - tasks[i].bt;
    printf("%-10d %-5d %-6d %-5d %-5d %-5d\n",
        tasks[i].id, tasks[i].bt, tasks[i].period, tasks[i].ct, tasks[i].wt, tasks[i].tat);
}

```

```

tasks[i].ct, tasks[i].wt, tasks[i].tat);
print-chart(schedule, time, "RMS");
double u = utilization-rms(tasks, n);
if (u < bound) printf("RMS Result: schedulable. (u = %.2f,
                    Bound = %.2f) \n", u, bound);
else printf("RMS Result: Not schedulable
            (u = %.2f, Bound = %.2f) \n", u, bound);
}
int main() {
    int n;
    printf("Enter number of processes:");
    scanf("%d", &n);
    struct Task tasks[n];
    printf("\n Enter process details: \n");
    for (int i = 0; i < n; i++) {
        tasks[i].id = i + 1;
        printf("\n Process %d: \n", i);
        printf("Burst time (for EDF):");
        scanf("%d", &tasks[i].deadline);
        printf("Period (for RMS):");
        scanf("%d", &tasks[i].period);
    }
    edf(tasks, n);
    rms(tasks, n);
    return 0;
}

```

output:

Enter process details:

Process 0;

Burst Time: 2

Deadline (For EDF): 5

Period (for RMS): 5

Process 1:

Burst Time: 2

Deadline (for EDF): 7

Period (for RMS): 5

Process 2:

Burst Time: 3

Deadline (for EDF): 8

Period (for RMS): 8

EDF Scheduling

CPU Utilization: 1.06

Not schedulable (utilization > 1)

Rate Monotonic Scheduling (RMS)

CPU ~~scheduling~~ utilization: 1.06

RM Bound: 0.7798

NOT schedulable (utilization > RM Bound)

Enter number of processes: 3

Enter process details:

Process 1:

Burst Time (for EDF): 3

Deadline (for EDF): 7

Period (for RMS): 20

Process 2:

Burst time (for EDF): 2

Deadline (for EDF): 4

Period (for RMS): 5

Process 3:

Burst time (for EDF): 2

Deadline (for EDF): 8

Period (for RMS): 10

EDF scheduling

Process ID	BT	Deadline	CT	WT	TAT
2	2	4	2	0	2
1	3	7	5	2	5
3	2	8	7		

EDF Gantt chart

| P₂ | P₂ | P₁ | P₁ | P₁ | P₃ | P₃ |
 0 1 2 3 4 5 6 7

EDF Result: Schedulable ($U=0.75$)

RMS

ProcessID	BT	Period	CT	WT	TAT
2	2	5	2	0	2
3	2	10	4	2	4
1	3	20	7	4	7

Gantt chart

| P₂ | P₂ | P₃ | P₃ | P₁ | P₁ | P₁ |
 0 1 2 3 4 5 6 7

RMS result: Schedulable ($U=0.75$,
 Bound = 0.78)

✓
 8/14

Q. Bounded Buffer

```

#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define SIZE 5

int buffer[SIZE];
int in=0, out=0;
sem_t empty, full;
pthread_mutex_t mutex;

void* producer (void* arg) {
    int item;
    for(int i=0; i<15; i++) {
        item = i;
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);
        buffer[in] = item;
        printf("Produced: %d at buffer [%d]\n",
               item, in);
        in = (in+1) % SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&full);
        usleep(100000);
    }
}

void* consumer (void* arg) {
    int item;
    for(int i=0; i<15; i++) {

```

```

sem_wait(&full);
pthread_mutex_lock(&mutex);
item = buffer[out];
printf("Consumed: %d from buffer[%d]\n",
       item, out);
out = (out+1) % SIZE;
pthread_mutex_unlock(&mutex);
sem_post(&empty);
usleep(150000);
}
}

```

```

int main() {
    pthread_t P, C;
    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);
    pthread_create(&P, NULL, producer, NULL);
    pthread_create(&C, NULL, consumer, NULL);

    pthread_join(P, NULL);
    pthread_join(C, NULL);
    return 0;
}

```

Output:-

Produced: 0 at buffer[0]
 Consumed: 0 from buffer[0]
 Produced: 1 at buffer[1]
 Produced: 1 from buffer[1]

Produced: 2 at buffer[2]
Consumed: 2 from buffer[2]

Produced: 3 at buffer[3]

Produced: 4 at buffer[4]

Consumed: 3 from buffer[3]

Produced: 5 at buffer[0]

Consumed: 4 from buffer[4]

Produced: 6 at buffer[1]

Produced: 7 at buffer[2]

Consumed: 5 from buffer[0]

Produced: 8 at buffer[3]

Consumed: 6 from buffer[1]

Produced: 9 at buffer[4]

Produced: 10 at buffer[0]

Consumed: 7 from buffer[2]

Produced: 11 at buffer[1]

Consumed: 8 from buffer[3]

Produced: 12 at buffer[2]

Consumed: 9 from buffer[4]

Produced: 13 at buffer[3]

Produced: 14 at buffer[4]

Tracing

BUFFER-SIZE = 3

Buffer = [-, -, -]

in = 0, out = 0

empty = 3, full = 0, mutex = 1

Step	Operation	Buffer State	in	out	empty	full
1	Producer inserts A	[A, -, -]	1	0	2	1
2	Producer inserts B	[A, B, -]	2	0	1	2
3	Producer inserts C	[A, B, C]	0	0	0	3
4	Producer inserts D	[-, B, C]	0	0	0	3
5	Consumer removes A	[D, B, C]	1	1	0	3
6	Producer inserts B	[D, -, C]	1	2	1	2
7	Consumer removes B	[D, -, C]	1	2	1	2

2. Dining philosopher

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define N 5
#define EAT-COUNT 5

pthread_mutex_t forks[N];

void * philosopher(void * num) {
    int id = *(int *) num;

    for (int i = 0; i < EAT-COUNT; i++) {
        pthread_mutex_lock(&forks[id]);
        printf("Philosopher %d picked up left fork\n", id, id);

        pthread_mutex_lock(&forks[(id+1)%N]);
        printf("Philosopher %d picked up right fork\n", id, (id+1)%N);

        printf("Philosopher %d is eating (meal %d).\n", id, i+1);

        pthread_mutex_unlock(&forks[id]);
        pthread_mutex_unlock(&forks[(id+1)%N]);
        printf("Philosopher %d put down forks %d and\n", id, id, (id+1)%N);
    }
}
```

```
return NULL;
```

```
}
```

```
int main() {
```

```
    pthread_t tid[N];
```

```
    int ids[N];
```

```
    for(int i=0; i<N; i++){
```

```
        pthread_mutex_init (&forks[i], NULL);
```

```
        ids[i]=i;
```

```
    }
```

```
    for (int i=0; i<N; i++){
```

```
        pthread_create (&tid[i], NULL, philosopher,  
                        &ids[i]);
```

```
    }
```

```
    for(int i=0; i<N; i++){
```

```
        pthread_join (tid[i], NULL);
```

```
    }
```

```
    return 0;
```

```
}
```

OK?

Bankers algorithm for purpose of deadlock avoidance.

```
#include <stdio.h>
#include <stdbool.h>

int main() {
    int n, m;
    printf("Enter number of processes");
    scanf("%d", &n);
    printf("Enter number of resources");
    scanf("%d", &m);
    int Allocation[n][m], Max[n][m], Need[n][m],
        Available[m];
    printf("Enter Allocation Matrix:\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            scanf("%d", &Allocation[i][j]);
    printf("Enter Maximum Demand Matrix:\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            scanf("%d", &Max[i][j]);
    printf("Enter Available Resources:\n");
    for (int j = 0; j < m; j++)
        scanf("%d", &Available[j]);
    for (int i = 0; i < n; i++)
```

```
for(int j=0; j<m; j++)
```

```
Need[i][j] = Max[i][j] - Allocation[i][j];
```

```
bool Finish[n];
```

```
for(int i=0; i<n; i++) Finish[i] = false;
```

```
int Work[m];
```

```
for(int j=0; j<m; j++) Work[j] = Available[j];
```

```
int SafeSeq[n], Count=0;
```

```
while (Count < n) {
```

```
    bool found = false;
```

```
    for(int i=0; i<n; i++) {
```

```
        if (!Finish[i]) {
```

```
            bool possible = true;
```

```
            for(int j=0; j<m; j++) {
```

```
                if (Need[i][j] > Work[j]) {
```

```
                    possible = false;
```

```
                    break;
```

```
                }
```

```
            }
```

```
            if (possible) {
```

```
                for(int j=0; j<m; j++)
```

```
                    Work[j] += Allocation[i][j];
```

```
                SafeSeq[Count++] = i;
```

```
                Finish[i] = true;
```

```
                found = true;
```

```
            }
```

```
        }
```

```
    }
```

```

if(!found) {
    printf("\nSystem is in an unsafe state.\n");
    return 0;
}
}

printf("\nSystem is in a Safe State.\nSafe
sequence is: ");

for(int i=0; i<n; i++) {
    printf("P%d", SafeSeq[i]);
    if (i != n-1) printf("->");
}
printf("\n");
return 0;
}

```

Output:-

Enter number of processes: 5

Enter number of resources: 3

Enter Allocation Matrix:

0 1 0

2 0 0

3 0 2

2 1 1

0 0 2

Enter Maximum Demand Matrix:

7 5 3

3 2 2

9 0 2

2 2 2

4 3 3

Enter Available Resources:

3 3 2

System is in a Safe state.

Safe Sequence is: $P_1 \rightarrow P_3 \rightarrow P_4 \rightarrow P_0 \rightarrow P_2$

Q. Deadlock detection

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, m, i, j, k;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter number of resources: ");
```

```
    scanf("%d", &m);
```

```
    int allocation[n][m], request[n][m], available[m];
```

```
    int finish[n];
```

```
    printf("Enter Allocation Matrix: \n");
```

```
    for(i=0; i<n; i++) {
```

```
        for(j=0; j<m; j++) {
```

```
            scanf("%d", &allocation[i][j]);
```

```
        }
```

```
    }
```

```
    printf("\nEnter Request Matrix: \n");
```

```
    for(i=0; i<n; i++) {
```

```
        for(j=0; j<m; j++) {
```

```
            scanf("%d", &request[i][j]);
```

```
        }
```

```
    }
```

```
Printf("\n Enter Available Resource: \n");
```

```
for(i=0; i<m; i++) {
```

```
scanf("%d", &request[i][0]);
```

```
}
```

```
for(i=0; i<n; i++) {
```

```
finish[i]=0;
```

```
}
```

```
int found;
```

```
do {
```

```
found=0;
```

```
for(i=0; i<n; i++) {
```

```
if (finish[i]==0) {
```

```
int canFinish=1;
```

```
for(j=0; j<m; j++) {
```

```
if (request[i][j] > available[j]) {
```

```
canFinish=0;
```

```
break;
```

```
}
```

```
}
```

```
if (canFinish) {
```

```
for(k=0; k<m; k++) {
```

```
available[k] += allocation[i][k];
```

```
}
```

```
finish[i]=1;
```

```
found=1;
```

```
} while(found);
```

```
int deadlock=0;
```

```
printf("\nProcesses In Deadlock:\n");
```

```
for(i=0; i<n; i++) {
```

```
    if(finish[i]==0) {
```

```
        printf("P%d", i);
```

```
        deadlock=1;
```

```
    }
```

```
}
```

```
if(deadlock==0) {
```

```
    printf("No deadlock detected. All processes  
        can finish.\n");
```

```
}
```

```
return 0;
```

```
}
```

Output:

Enter number of processes: 5

Enter number of resources: 3

Enter Allocation matrix:

0 1 0

2 0 0

3 0 3

2 1 1

0 0 2

Enter Request Matrix:

0 0 0

2 0 2

0 0 0

1 0 1

0 0 2

Enter Available Resources:

0 0 0

Processes in Deadlock:

No deadlock detected. All processes can finish.

S/S
S/S/26

Q. Given 6 memory partitions of

Q. Write a C program to simulate the following
Contiguous memory allocation techniques

a) Worst-fit

b) Best-fit

c) First-fit

Q. 300KB, 600KB, 350KB, 200KB, 750KB, 125KB (in order)
how would best fit, first fit, worst fit algorithms
place the processes of size 115KB, 500KB, 358KB,
200KB, 375KB

1. #include <stdio.h>

```
void firstFit (int blockSize[], int m, int processSize[],  
              int n) {
```

```
    int allocation[n];
```

```
    for (int i = 0; i < n; i++)
```

```
        allocation[i] = -1;
```

```
    int tempBlock[m];
```

```
    for (int i = 0; i < m; i++)
```

```
        tempBlock[i] = blockSize[i];
```

```
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < m; j++) {
```

```
            if (tempBlock[j] >= processSize[i]) {
```

```
                allocation[i] = j;
```

```
tempBlock[j] -= processSize[i];
```

```
break;
```

```
}
```

```
}
```

```
}
```

```
printf("\n---FirstFit---\n");
```

```
printf("Process No. | Process Size | Block No. | \n");
```

```
for (int i = 0; i < n; i++) {
```

```
    printf("%d | %d | %d | \n", i+1, processSize[i],
```

```
    if (allocation[i] != -1)
```

```
        printf("%d | \n", allocation[i]+1);
```

```
    else
```

```
        printf("Not Allocated | \n");
```

```
}
```

```
}
```

```
void bestFit (int blockSize[], int m, int  
              processSize[], int n) {
```

```
    int allocation[n];
```

```
    for (int i = 0; i < n; i++)
```

```
        allocation[i] = -1;
```

```
    int tempBlock[m];
```

```
    for (int i = 0; i < m; i++)
```

```
        tempBlock[i] = blockSize[i];
```

```
    i++) {
```

```
int bestIdx = -1;
```

```
for(int j=0; j<m; j++){
```

```
    if (tempBlock[j] >= processSize[i]){
```

```
        if (bestIdx == -1 || tempBlock[j] < tempBlock[bestIdx])
```

```
            bestIdx = j;
```

```
    }
```

```
}
```

```
if (bestIdx != -1) {
```

```
    allocation[i] = bestIdx;
```

```
    tempBlock[bestIdx] -= processSize[i];
```

```
}
```

```
}
```

```
printf("\n--- Best Fit ---\n");
```

```
printf(" Process No. \t Process Size \t Block No.\n");
```

```
for(int l=0; l<n; l++){
```

```
    printf("%d\t\t\t\t\t", l+1, processSize[l]);
```

```
    if (allocation[l] != -1)
```

```
        printf("%d\n", allocation[l]+1);
```

```
    else
```

```
        printf("Not Allocated\n");
```

```
}
```

```
}
```

```
void worstFit(int blockSize[], int m, int
```

```
    processSize[], int n) {
```

```
    int allocation[n];
```

```
    for(int i=0; i<n; i++){
```

```
        allocation[i] = -1;
```

```
int tempBlock[m];
```

```
for (int i=0; i<m; i++)
```

```
tempBlock[i] = blockSize[i];
```

```
for (int i=0; i<n; i++) {
```

```
int worstIdx = -1;
```

```
for (int j=0; j<m; j++) {
```

```
if (tempBlock[j] >= processSize[i]) {
```

```
if (worstIdx == -1 || tempBlock[j] >
```

```
tempBlock[worstIdx])
```

```
worstIdx = j;
```

```
}
```

```
}
```

```
if (worstIdx != -1) {
```

```
allocation[i] = worstIdx;
```

```
tempBlock[worstIdx] -= processSize[i];
```

```
}
```

```
}
```

```
printf("\n--- Worst Fit ---\n");
```

```
printf("Process No. | Process Size | Block No. | \n");
```

```
for (int i=0; i<n; i++) {
```

```
printf("%d | %d | %d | \n", i+1, processSize[i],
```

```
if (allocation[i] != -1)
```

```
printf("%d | \n", allocation[i]+1);
```

```

printf("Not Allocated\n");
}
}

int main() {
    int m, n;
    printf("Enter number of memory blocks:");
    scanf("%d", &m);
    int blockSize[m];
    printf("Enter Sizes of %d memory blocks:\n", m);

    for(int i=0; i<m; i++)
        scanf("%d", &blockSize[i]);

    printf("Enter number of processes:");
    scanf("%d", &n);
    int processSize[n];
    printf("Enter Sizes of %d processes:\n", n);
    for(int i=0; i<n; i++)
        scanf("%d", &processSize[i]);

    firstFit(blockSize, m, processSize, n);
    bestFit(blockSize, m, processSize, n);
    worstFit(blockSize, m, processSize, n);

    return 0;
}

```

Output:-

Enter number of memory blocks: 5

Enter sizes of 5 memory blocks:

100 500 200 300 400

Enter number of Processes 4

Enter size of 4 processes:

212 417 112 426

--- First Fit ---

Process No.	Process Size	Block No.
1	212	2
2	417	5
3	112	2
4	426	Not Allocated

--- Best Fit ---

Process No.	Process Size	Block No.
1	212	4
2	417	2
3	112	3
4	426	5

--- Worst Fit ---

Process No.	Process Size	Block No.
1	212	5
2	417	2
3	112	5
4	426	Not Allocated

✓

22/5/26

Q. write a program for page replacement algorithms

- a) FIFO
- b) LRU
- c) Optimal

```
#include <stdio.h>
```

```
void fifo(int p[], int n, int f) {  
    int fr[10], i, j, k, pos = 0, fault = 0, hit;  
    for(i = 0; i < f; i++) fr[i] = -1;  
    printf("\nFIFO:\n");  
    for(i = 0; i < n; i++) {  
        hit = 0;  
        for(j = 0; j < f; j++)  
            if (fr[j] == p[i]) hit = 1;  
        if (!hit) {  
            fr[pos] = p[i];  
            pos = (pos + 1) % f;  
            fault++;  
        }  
        for(k = 0; k < f; k++)  
            printf("%d", fr[k]);  
        printf("\n");  
    }  
    printf("Page Faults = %d\n", fault);  
}
```

```
void lru(int p[], int n, int f) {
```

```
int fr[10], t[10], i, j, k, pos, min, cnt = 0;
```

```
fault = 0, hit;
```

```
for (i = 0; i < f; i++) fr[i] = t[i] + 1;
```

```
printf("\nLRU: |n");
```

```
for (i = 0; i < n; i++) {
```

```
    hit = 0;
```

```
    for (j = 0; j < f; j++) {
```

```
        if (fr[j] == p[i]) {
```

```
            cnt++;
```

```
            t[j] = cnt;
```

```
            hit = 1;
```

```
        }
```

```
    }
```

```
    if (!hit) {
```

```
        min = 0;
```

```
        for (j = 1; j < f; j++)
```

```
            if (t[j] < t[min])
```

```
                min = j;
```

```
        fr[min] = p[i];
```

```
        cnt++;
```

```
        t[min] = cnt;
```

```
        fault;
```

```
    }
```

```
for (k=0; k<f; k++)
```

```
    printf("%d", fr[k]);
```

```
    printf("\n");
```

```
}
```

```
printf("Page Faults = %d\n", fault);
```

```
}
```

```
int main() {
```

```
    int P[50], n, f, i;
```

```
    printf("Enter no of pages:");
```

```
    scanf("%d", &n);
```

```
    printf("Enter pages:");
```

```
    for (i=0; i<n; i++)
```

```
        scanf("%d", &P[i]);
```

```
    printf("Enter no of frames:");
```

```
    scanf("%d", &f);
```

```
    fifo(p, n, f);
```

```
    lru(p, n, f);
```

```
    optimal(p, n, f);
```

```
    return 0;
```

```
}
```

output:-

Enter no of pages: 8

Enter Pages: 1 2 3 4 1 2 5 1

Enter no of frames: 3

FIFO:

1 -1 -1

1 2 -1

1 2 3

4 2 3

4 1 3

4 1 2

5 1 2

5 1 2

Page Faults = 7

LRU:

1 -1 -1

1 2 -1

1 2 3

4 2 3

4 1 3

4 1 2

5 1 2

5 1 2

Page Faults = 7

Optimal:

1 -1 -1

4 2 -1

1 2 3

1 2 4

1 2 4

1 2 4

1 5 4

1 5 4

Page Faults = 5

850
212 15 12 6